MixBytes()

[SECURITY AUDIT REPORT]

BarterLab BarterLend

PREPARED FOR BARTERLAB

REPORT DATE

October 30, 2026

[CONTENTS]

Table of Contents

1. Introduction 4
   1.1 Disclaimer 4
   1.2 Executive Summary 4
   1.3 Project Overview 6
   1.4 Security Audit Methodology 11
   1.5 Risk Classification 13
   1.6 Summary of Findings 14
2. Findings Report 16
   2.1 Critical 16
      C-1 Tick share inflation attack 16
   2.2 High 18
      H-1 Positions can be opened at the liquidation edge with HF=1 18
   2.3 Medium 20
      M-1 Borrow queue clogging via the self-skip mechanism 20
      M-2 liquidate() has no loss and deadline protection 22
      M-3 Underlying pool incentive rewards are never claimed 24
      M-4 Bucket and tick aggregates drift apart and cause DoS 25
      M-5 One protocolFee is applied to two different bases, so the same move costs (1 + fee)× more through reloan than through borrow 26
      M-6 NoDebtToLiquidate blocks bad-debt socialisation when discounted collateral recovery floors to zero 27
   2.4 Low 29
      L-1 RWA pitfalls 29
      L-2 Underlying-pool utilization DoS 30
      L-3 Unbounded getUserMarkets() registry can be inflated by anyone 32
      L-4 The rollover keeper controls direction, counterparty, timing, and fee recipient 33
      L-5 _creditProvidedLoan credits owner fee shares without registering the market 35
      L-6 _previewProvide does not model LoanPool burns across Reloan fills, so preview understates fillability 36
      L-7 _previewProvide does not accumulate borrower debt across fills, so ExactOut preview can overstate fillability 37

L-8 Borrow-order invalidation depends on counterparty size and can cancel a still-collateralized remainder 39

3. About MixBytes 41

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

BarterLend is a fixed-term, fixed-yield lending protocol in which lenders and borrowers meet through on-chain APR orderbooks rather than a single pooled rate. Each CreditPool is deployed from an immutable factory that pins facet bytecode and whitelisted underlying yield adapters, routing idle liquidity into Aave V3 or Morpho Blue markets while borrowers post collateral against time-bucketed maturities. Core actors include pool owners (administrative configuration), lenders and borrowers (order placement and loan lifecycle), liquidator bots, and an optional rollover extension that automates debt rollovers across maturing markets. Price discovery for collateral health checks relies on a Chainlink-based oracle, with Permit2 used for gas-efficient token approvals on selected flows.

The review examined both the core CreditPool accounting and order-matching logic and its on-chain integrations with underlying yield venues, Chainlink price feeds, and Permit2. Auditors also applied the standard internal checklist covering access control, reentrancy, integer rounding, oracle staleness, delegatecall safety, and economic edge cases around liquidations and market scheduling.

Aave V3 Pool, Morpho Blue, and the Adaptive Curve IRM contracts were outside the audit scope; assessment was limited to how the in-scope adapters invoke them and to published interface assumptions. Chainlink aggregator contracts and the Uniswap Permit2 contract were likewise excluded from line-by-line review, with coverage confined to integration points visible in in-scope code.

Key notes and recommendations

- It is important to note that this audit assumed borrow/supply assets are standard ERC-20 tokens, without transfer fees or rebasing. RWA tokens may be supported by the protocol, but only with a correctly implemented oracle; see **RWA pitfalls**.
- Protocol fee is only bounded below 100% (`fee < UNIT`). We recommend capping the protocol fee to a reasonable max.
- The liquidation grace period has no lower bound, so a zero value makes any position term-liquidatable the instant its market matures, removing the intended post-maturity repay/rollover window.
- A separate `borrowLTV` is enforced on debt-increasing paths, but the code enforces neither a gap between `borrowLTV` and `liquidationLTV` nor a lower bound on `borrowLTV` . If they are configured equal, a position can be opened right at the liquidation line and liquidated on ordinary oracle noise, so a safe margin relies on the admin configuration.

- Each `CreditPool` can be assigned any owner at creation, and its owner can change fees and liquidation parameters for new markets as well as order and collateral limits, so users should check `owner()` and the configuration of each pool; we recommend assigning pool ownership to a multisig.

1.3 Project Overview

Summary

CLIENT	BarterLab
CATEGORY	Lending
PROJECT	BarterLend
TYPE	Solidity
PLATFORM	EVM
TIMELINE	06.08.2026 - 29.10.2026

Scope of Audit

`src/ChainlinkPriceOracle.sol` `src/CreditPool.sol` `src/CreditPoolBase.sol`
`src/CreditPoolFactory.sol` `src/CreditPoolRolloverExtension.sol` `src/Delegator.sol`
`src/adapters/AaveV3Adapter.sol` `src/adapters/MorphoMarketV1AdaptiveCurveAdapter.sol`
`src/facets/CreditPoolFacet1.sol` `src/facets/CreditPoolFacet2.sol`
`src/facets/CreditPoolFacet3.sol` `src/facets/CreditPoolFacet4.sol` `src/lib/BitSet.sol`
`src/lib/MarketsConfig.sol` `src/lib/PriceMath.sol` `src/lib/Set.sol`

Versions Log

DATE	COMMIT HASH	NOTE
06.08.2026	`677c7da9279a2e3e90da5e74a72f62c0b16e2cd8`	Initial commit (private repository)
28.10.2026	`126035e9e922d80f5676e024d036f5cb0bcb4da5`	Final re-audit commit (private repository)
28.10.2026	`546b5cdd75f1551336e0608eb201d9714ff14347`	Public repository commit, identical to private commit 126035e9e922d80f5676e024d036f5cb0bcb4da5
29.10.2026	`074f2b975431a13bfec75640aa65c1c2ffa8595c`	Deployment commit (public repository)

The audit was conducted on a private repository, BarterLab/BarterLend-priv (formerly BarterLab/BarterLend): initial commit 677c7da, final re-audit commit 126035e. The audited code was then published to the public repository BarterLab/BarterLend as commit 546b5cd, which is identical to private commit 126035e.

Commit 074f2b9 was subsequently added on top of 546b5cd in the public repository only. It removes the `10% * (1 - liquidationLTV)` cap from `liquidationDiscountMin`, which is now bounded only by `liquidationDiscountMax`. The effective liquidation discount is still capped by `liquidationDiscountMax <= 40% * (1 - liquidationLTV)`, so its upper bound is unchanged; the change only allows a higher owner-configured discount for positions that have just become liquidatable. No new issues were identified in this commit.

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
CreditPoolFactory.sol	0xDd993FF5d323499388Ec316D1286BdE639C76670	Monad
CreditPoolRolloverExtension.sol	0x35867B231060249a6895489a5532d36c5Ecb2bef	Monad
AaveV3Adapter.sol	0x430284e6Bd016103ABae19b4C0ae9BF1895420Ba	Monad
MorphoMarketV1AdaptiveCurveAdapter.sol	0x6D7f5055FA759452C991F23549eA632F8e5FCF8e	Monad
CreditPoolFacet1.sol	0xfAf011d17c1D41A77693487590d444C1b5d46995	Monad
CreditPoolFacet2.sol	0xb50AcDC09D9E9D936c28854e3D20bbdDF85F44dE	Monad
CreditPoolFacet3.sol	0x73d00e5367F6F69F586dDF5d10a052C7C911759f	Monad
CreditPoolFacet4.sol	0xc94E35Adc5ebB7B6F5f578B291d805403cf99F14	Monad

The deployment was reviewed against 074f2b97, block `117623973`.

Verified on-chain source for all eight deployed contracts matches the local repository byte-for-byte: the audited contracts (`CreditPoolFactory`, `CreditPoolRolloverExtension`, `AaveV3Adapter`, `MorphoMarketV1AdaptiveCurveAdapter`, `CreditPoolFacet1` to `CreditPoolFacet4`, `CreditPoolBase`) together with all their dependencies, namely the project interfaces and libraries (`ICreditPool`, `ICreditPoolExtension`, `ICreditPoolFactory`, `ICreditPoolRolloverExtension`, `IUnderlyingPool`, `IPriceOracle`, `ILiquidator`, `IAaveV3AToken`, `BitSet`, `MarketsConfig`, `PriceMath`, `Set`), OpenZeppelin Contracts, Uniswap Permit2 interfaces, Aave V3 Origin interfaces, and Morpho Blue and Morpho Blue IRM libraries. All contracts were compiled with solc `0.8.29`, optimizer enabled with 200 runs and EVM version `osaka`, matching `foundry.toml`. The runtime code of `CreditPoolRolloverExtension` also equals the local build after masking immutables and metadata.

`CreditPool` and `ChainlinkPriceOracle` are not listed in the table because they are deployed per market rather than once per protocol: `CreditPoolFactory.createPool` deploys a new `CreditPool` for each market, and a separate `ChainlinkPriceOracle` instance is deployed for each pair of price feeds, so further instances of both will be deployed as new markets are added.

The four facets were deployed by `CreditPoolFactory` through `deployFacet` from creation code cached in `CodeChunk` contracts, in the transactions recorded by `FacetDeployed`, and are verified on Monadscan as described above. The deployed facets are shared by all pools created through this factory: `createPool` passes the four addresses returned by `facets()` to every new `CreditPool`, which delegates calls to them, and no per-pool facet copies are deployed. `facets()` returns the four verified facet addresses in order.

`CreditPool` (including `Delegator`) is audited and has no single deployment address; each pool is deployed by `createPool` from creation code cached in the factory. The cached creation code for all five code IDs (`CreditPool` and the four facets) was read from factory storage and compared with the local build: the executable bytecode is identical, and the only difference is the IPFS hash inside the trailing CBOR metadata, which depends on compilation input paths and has no effect on execution. For each facet, the metadata hash in

the cached creation code equals the one in the runtime code deployed at the facet address, which ties the cached code to the verified facet sources. `configurationFrozen` is `true`, so cached code, facets and approved adapters can no longer be changed, and every pool created by the factory (`isPool(pool) == true`) runs the audited `CreditPool` code without modification. Market-specific parameters, such as the asset and collateral tokens, price oracle, Permit2 address, underlying pool configuration, LTVs, liquidation discounts, fees, caps and market schedule, are supplied in `createPool` at deployment. The factory only requires the underlying pool adapter to be one of the approved adapters with the pinned code hash; the pool constructor checks the remaining parameters for internal consistency only (for example `borrowLTV <= liquidationLTV < 100%`, the liquidation discount and grace period bounds, `protocolFee < 100%`, and an oracle whose `source` and `target` match the collateral and asset tokens), not for economic soundness.

Each `CreditPool` has its own `owner`, set from `InitializationParams.owner` at creation, which may differ from the factory owner. The pool owner can change selected parameters (`setProtocolFee`, `setTotalCollateralCap`, `setBorrowOrderMinAmount`, `setProvideOrderMinAmount`, `setMaxActiveProvideOrders`, `setMaxActiveBorrowOrders`, `setMinDebtRemainingAfterLiquidation`, `setLiquidationDiscountMin`, `setLiquidationDiscountMax`, `setLiquidationGracePeriod`), call `settleUnderlyingPoolDust` and transfer pool ownership. Changes to the protocol fee, liquidation discounts and grace period do not affect markets that are already active, while the other parameters take effect immediately. The pool owner cannot change the pool code, facets, underlying pool adapter, price oracle, tokens, LTVs or market schedule. `createPool` accepts any non-zero `owner`, and the pool owner can later transfer ownership without involving the factory owner, so the owner of a given pool is not guaranteed to be the factory Safe. Users should therefore check that a pool is registered in the factory (`isPool`) and review its `owner()` and configuration before interacting with it. We recommend assigning pool ownership to a multisig rather than an EOA.

Configured integrations were verified on-chain. `MorphoMarketV1AdaptiveCurveAdapter` resolves `MORPHO` to the verified `Morpho` contract `0xD5D960E8C380B724a48AC59E2DfF1b2CB4a1eAee` and `ADAPTIVE_CURVE_IRM` to the verified `AdaptiveCurveIrm` `0x09475a3D6eA8c314c592b1a3799bDE044E2F400F`, whose own `MORPHO()` points to the same Morpho instance. `AaveV3Adapter` is stateless and takes the reserve from per-pool configuration. The factory approves exactly two underlying pool adapters, the deployed `AaveV3Adapter` and `MorphoMarketV1AdaptiveCurveAdapter`, and the code hash pinned for each equals its current on-chain code hash. `CreditPoolRolloverExtension` resolves `PERMIT2` to the canonical Permit2 deployment `0x000000000022D473030F116dDEE9F6B43aC78BA3` and `FACTORY` to the deployed `CreditPoolFactory`.

The deployed code corresponds to commit `074f2b9`, which follows the re-audit commit `546b5cd` and only relaxes the `liquidationDiscountMin` bound in `CreditPoolBase`; this change is described in the Versions Log and the deployed facets and cached pool code contain it.

On-chain privileged roles resolve as follows, with each role and the contract it controls listed below:

ROLE	ADDRESS	WHAT THIS IS
`CreditPoolFactory.owner`	0x174DA5aB9C0E79504c87727C061566866D38ff47	Gnosis Safe v1.5.0 multisig (SafeL2 singleton), 3-of-4 (75%), all owners EOAs, no modules, no guard
`CreditPoolFacet1-4.owner (facet own storage)`	0x0000000000000000000000000000000000000000	Zero address; facets are delegatecall targets and pool ownership lives in each Credit Pool's own storage
`AaveV3Adapter, MorphoMarketV1Adaptive CurveAdapter, CreditPoolRollover Extension`	N/A	No privileged roles; configuration is fixed by immutables or constructor-set state

The factory owner Safe is the only privileged role among the contracts listed above. Its signers are 0x4C39790C3bf761D58E07156A0B24806F6a3b729d, 0x96D19072b68d88625414Bd9e7c2e54f6D6778908, 0x2D8AFE529245b3184984FE9De14c1eecBA71e196 and 0xF1839b62E98a253b4f1367aBcAE48d9263637580, all EOAs with no code, so there are no nested Safes or controller contracts on the ownership path. The Safe proxy points to singleton 0xEdd160fEBBD92E350D4D398fb636302fccd67C7e (`SafeL2` , `VERSION() = 1.5.0`). `getModulesPaginated` returns no modules, and both the transaction guard slot and the module guard slot are zero. The fallback handler is 0x3EfCBb83A4A7AfcB4F68D501E2c2203a38be77f4; its source is not verified on Monadscan, but it answers `getMessageHash(bytes)` through the Safe, which is consistent with the standard Safe `CompatibilityFallbackHandler` . The fallback handler serves calls to selectors not implemented by the Safe, such as token receiver callbacks and ERC-1271 signature validation; it is invoked via `CALL` and cannot change Safe state or execute transactions. Pool owners are set per pool at creation, as described above, and are not part of this table.

The factory was initially owned by the deployer EOA 0x413B3323cFB219567A606b51b49f839D4C052e17, which performed the configuration (creation code caching, facet deployment, adapter approval, `freezeConfiguration`) and then transferred ownership to the Safe in transaction 0x95594a6a8c217eb4fc89b8f53045027b56925ba57b78aed51a427525ab449094 at block 117460072. The deployer holds no remaining role on the verified contracts. Since `configurationFrozen` is `true` , the factory owner can no longer change cached code, facets or approved adapters; the remaining owner capabilities are `createPool` , `transferOwnership` and `renounceOwnership` , the last of which would permanently disable pool creation through this factory.

All eight deployed contracts match the reviewed commit, the factory-deployed pool and facet code matches the local build apart from compiler metadata, external integrations point to the expected Morpho and Permit2 contracts, and the only privileged role on the contracts listed above is held by a 3-of-4 Safe with EOA signers, no modules and no guard.

The Safe fallback handler is not verified on Monadscan. The fallback handler serves calls to selectors not implemented by the Safe, such as token receiver callbacks and ERC-1271 signature validation; it is invoked via CALL and cannot change Safe state or execute transactions. This is not critical for BarterLend deploy: `CreditPoolFactory` and `CreditPool` authorize the owner by `msg.sender == owner` rather than by signatures, so a forged ERC-1271 signature cannot be used to call owner-only functions. The exposure is limited to assets held by the Safe itself, if it grants approvals to protocols that accept ERC-1271 signatures.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2 Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● FIXED - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● PARTIALLY FIXED - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● ACKNOWLEDGED - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count

1	1	6	8	16
CRITICAL	HIGH	MEDIUM	LOW	TOTAL

Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

ID	FINDING	SEVERITY	STATUS
C-1	Tick share inflation attack	CRITICAL	FIXED
H-1	Positions can be opened at the liquidation edge with HF=1	HIGH	FIXED
M-1	Borrow queue clogging via the self-skip mechanism	MEDIUM	FIXED
M-2	liquidate() has no loss and deadline protection	MEDIUM	FIXED
M-3	Underlying pool incentive rewards are never claimed	MEDIUM	ACKNOWLEDGED
M-4	Bucket and tick aggregates drift apart and cause DoS	MEDIUM	FIXED
M-5	One protocolFee is applied to two different bases, so the same move costs (1 + fee)× more through reloan than through borrow	MEDIUM	FIXED
M-6	NoDebtToLiquidate blocks bad-debt socialisation when discounted collateral recovery floors to zero	MEDIUM	FIXED
L-1	RWA pitfalls	LOW	ACKNOWLEDGED
L-2	Underlying-pool utilization DoS	LOW	ACKNOWLEDGED
L-3	Unbounded getUserMarkets() registry can be inflated by anyone	LOW	ACKNOWLEDGED

ID	FINDING	SEVERITY	STATUS
L-4	The rollover keeper controls direction, counterparty, timing, and fee recipient	LOW	ACKNOWLEDGED
L-5	_creditProvidedLoan credits owner fee shares without registering the market	LOW	ACKNOWLEDGED
L-6	_previewProvide does not model LoanPool burns across Reloan fills, so preview understates fillability	LOW	FIXED
L-7	_previewProvide does not accumulate borrower debt across fills, so ExactOut preview can overstate fillability	LOW	ACKNOWLEDGED
L-8	Borrow-order invalidation depends on counterparty size and can cancel a still-collateralized remainder	LOW	FIXED

2.1 Critical

C-1 Tick share inflation attack

CRITICAL

FIXED

DESCRIPTION

Each provide-order tick is an ERC4626-style share pool: a lender's redeemable assets are `tickAssets * order.shares / tick.totalShares`. Shares are minted with **ceil** rounding (`Math.Rounding.Ceil`); the first deposit into an empty tick mints `assets * SCALE`.

An attacker who is the sole provider in a tick can collapse `totalShares` down to `1 wei` while keeping the tick's per-share value large, using repeated **donate → withdraw** cycles:

1. **Donate** assets directly into the pool's underlying (Aave) position. `totalAssets()` is read as the raw aToken balance, so an unsolicited transfer inflates it and is credited to the attacker's tick through the underlying-pool index.
2. **Cancel/withdraw** the tick back down to the minimum residual balance. A withdrawal burns shares in proportion to the assets removed, so the larger the donation, the bigger the share reduction — `totalShares` drops to ~`residual / (residual + donation)` of its previous value.

The donation only needs to increase the value that the adapter's `totalAssets(creditPool, config)` reports; both backends expose a permissionless way to do so:

- **Aave (`AaveV3Adapter`)** — `totalAssets` reads the raw `aToken.balanceOf(creditPool)`. Since aTokens are freely transferable ERC20 tokens, the attacker simply transfers aTokens to the credit-pool address (`aToken.transfer(creditPool, amount)`), or supplies the underlying on the pool's behalf (`aavePool.supply(asset, amount, creditPool, 0)`) to mint aTokens straight to it. No interaction with the credit pool is required.
- **Morpho (`MorphoMarketV1AdaptiveCurveAdapter`)** — `totalAssets` reads the credit pool's Morpho supply position: `position(id, creditPool).supplyShares` converted with `toAssetsDown(...)`. There is no ERC20 to transfer, but Morpho's `supply` accepts an `onBehalf` argument, so the attacker calls `MORPHO.supply(marketParams, amount, 0, creditPool, "")` to mint supply shares directly to the credit pool's position, inflating its reported assets.

In both cases the extra assets are attributed to the shared underlying-pool index (not to any specific market) and thus flow into the attacker's tick when it dominates the pool's active assets.

One cycle example:

1. Create a p-order of 1.00 ETH; this mints `1e18 * 1e27 = 1e45` shares.
2. Donate another 9.00 ETH into the Aave position. The tick balance is now 10.0 ETH; the share count is unchanged.
3. Withdraw 9.00 ETH, leaving the original 1.00 ETH. The share count drops from `1e45` to `1e44` — a 90% reduction (10×) — so the price of a single share is multiplied by 10.

The `provideOrderMinAmount` guard only bounds the residual, so it slows the attack but does not prevent it. Under the default `provideOrderMinAmount = 1e18` each cycle reduces the share count by only ~101×, so it takes just a couple of dozen transactions to reach a state where the tick holds a single

share (`totalShares = 1 wei`) backing a non-zero asset amount. From there, every later depositor can be drained.

An attack example:

1. The tick starts at `totalShares = 1 wei` and `totalAssets = 1 ETH` , with the attacker owning that single share.
2. The victim deposits 100 ETH. Minting is `ceil(1 * 100 / 1) = 100` shares, so the tick now holds 101 ETH and 101 wei shares.
3. The hacker provides 1 ETH + 1 wei. Minting is `ceil(101 * (1e18 + 1) / 101e18) = 2` shares, although the fair value of that deposit is only ~1 share. Together with the share they already held, the hacker now owns 3 of the 103 shares, redeemable for ~3 ETH, while having spent only ~2 ETH in total — a ~1 ETH profit taken directly from the victim's deposit.
4. The attacker repeats step 3, each time minting more rounded-up shares and capturing a growing share of the victim's deposit.

Once the share price is inflated, the attack can be applied throughout the entire lifetime of the market.

RECOMMENDATION

First and foremost, we recommend addressing the root cause:

- When placing/previewing a p-order, round shares **down**, in the protocol's favor: charge the user more assets and mint fewer shares.

This naive fix, however, opens up other problems:

- if the share price exceeds the deposit amount, `floor` yields `0` and reverts (`ZeroSharesMinted`), which lets an attacker permanently DoS the tick;
- it enables the opposite attack (against non-first depositors) via `floor` rounding, where both the attacker and the victim are minted a single share (see Rounding to one share example).

Both problems are solved by:

1. Capping the underlying yield growth on the adapter via `MAX_APR` .
2. Applying the OpenZeppelin virtual-offset pattern: `shares + VIRTUAL_SHARES` , `assets + VIRTUAL_ASSETS` .

Regarding the OZ virtual offset, we recommend:

- using `VIRTUAL_SHARES = SCALE` , `VIRTUAL_ASSETS = 1` ;
- removing the special-case branch `if (tick.totalShares == 0) return assets * SCALE` , since with the virtual offset it collapses into the general formula anyway: `assets.mulDiv(0 + VIRTUAL_SHARES, 0 + VIRTUAL_ASSETS) = assets.mulDiv(0 + SCALE, 0 + 1) = assets * SCALE` ;
- applying the offset consistently across mint/burn/withdraw/redeem/preview/balance, otherwise the accounting will desynchronize.

2.2 High

H-1 Positions can be opened at the liquidation edge with HF=1

HIGH

FIXED

DESCRIPTION

The protocol exposes a single `_liquidationLTV` parameter and uses it for both the borrow-time health check and the liquidation trigger. This means a position can be opened right on the liquidation edge at HF=1, so even a 1-wei drop in collateral price leads to user liquidation.

There is no lower `maxBorrowLTV` (or equivalent) that would create a margin between the maximum debt a user can take on and the threshold at which liquidation starts.

Problem 1: Rollover can push a healthy position onto the liquidation edge

The only borrower-side bound on a rollover is the aggregate `maxDebtFactor`; the refinance leg itself hard-codes `maxDebt = type(uint128).max`, so per-leg slippage/APR protection is off and the keeper freely chooses the destination tick — including its own resting order at a punitive APR:

```
if (factor > maxDebtFactor) revert MaxRolloverFactorExceeded(factor, maxDebtFactor);
```

`maxDebtFactor` is neither validated nor bounded at `setRolloverConfig`, and the grace ramp relaxes the effective factor toward the deadline. So a borrower who opts into rollover with a loose (or unbounded) factor can be rolled onto the keeper's `MAX_APR` tick and sized so post-roll health lands just above `UNIT` (`HF_after = HF_before / (1 + factor)`). In `test/CreditPool/88_rollover_pushes_victim_to_the_edge.t.sol` a borrower resting comfortably at `HF = 1.5` is moved to `HF ≈ 1.0013` and liquidated on the first `0.2%` oracle tick by the same keeper acting as lender and liquidator, seizing ~50% of the debt — even though the borrower took no risk beyond enabling rollover.

Problem 2: `borrow()` has no deadline, so a stalled order can fill right at the liquidation edge

A user can submit a `borrow()` whose order rests in the queue and is filled later, when market conditions have already moved. Because `borrow()` has no `deadline` (or freshness) parameter, there is nothing that invalidates a stale order: it can execute at `HF = 1` and the position is opened exactly on the liquidation line. With the shared `_liquidationLTV`, the very next oracle tick — even a 1-wei collateral price drop — makes the freshly opened position immediately liquidatable, without the borrower taking any risk beyond placing the order.

Other Concerns

1. **No safety buffer for borrowers.** A position can be opened right at the liquidation boundary. Ordinary oracle noise, a stale price tick, or a small market move between order placement and fill can push `HF` below `UNIT` immediately after the borrow, exposing the user to liquidation without any intentional risk-taking beyond the configured LTV.
2. **High per-position leverage.** Because the same LTV governs both borrowing and liquidation, a user with a fixed amount of equity can scale absolute position size (collateral and debt) up to the liquidation limit. On less liquid collateral markets this increases the notional that must be sold

during liquidation; a liquidator may struggle to unwind an oversized position within available DEX/OTC liquidity, increasing bad-debt risk for lenders even when the position is technically liquidatable.

3. **Potential composability risk.** The absence of a borrow/liquidation LTV gap may also enable downstream on-chain attack paths that rely on opening positions at the exact liquidation edge (e.g. in combination with oracle updates, mempool timing, or rollover/refinance flows). We did not identify a concrete exploit in the reviewed codebase, but the shared threshold leaves less room to absorb such interactions than a standard two-tier LTV design.

RECOMMENDATION

> We recommend introducing a separate, lower `maxBorrowLTV` (or `borrowLTV`) and enforcing it on every path that increases debt or posts collateral against new borrowing — including order fills, direct `borrow()`, and refinance/rollover legs — while keeping `_liquidationLTV` as the liquidation threshold. This gives borrowers an oracle-noise buffer so that a position opened at the borrow limit is not immediately liquidatable, and it protects orders that fill after a delay from executing right at the liquidation line.
>
> Additionally, aggregate exposure can be bounded with pool- or market-level collateral/debt caps; this addresses total market size but does not by itself give borrowers an oracle-noise buffer or cap per-position leverage relative to equity. The two mechanisms are complementary rather than interchangeable.
>
> Alternatively, if `borrowLTV = liquidationLTV`, for the rollover path specifically, we recommend introducing a `minHF` parameter: the borrower specifies the minimum health factor a rolled position must retain, and the refinance leg reverts if the post-roll `HF` would fall below it. This gives the borrower a real solvency margin that the keeper cannot size away, independent of `maxDebtFactor` and the grace ramp.
>
> Similarly, if `borrowLTV = liquidationLTV`, we recommend a `minHF` guard for borrow paths as well.

CLIENT'S COMMENTARY

MixBytes: A separate borrow LTV was introduced and enforced on every debt-increasing / collateral-decreasing path.

What is currently handled via admin setup trust, but can be solved at the code level:

1. **No enforced gap between `borrowLTV` and `liquidationLTV`.**
2. **No lower bound on `borrowLTV`.**

2.3 Medium

M-1 Borrow queue clogging via the self-skip mechanism

MEDIUM

FIXED

DESCRIPTION

The borrow-order minimum (`_borrowOrderMinAmount`) is enforced only on the placement and cancellation. The fill path does not re-check it: on a partial fill, `_provide()` simply decrements the order amount without validating the residual.

As a result, a Borrow order can be left with a residual down to 1 wei.

On its own this only produces a single 1 wei order at the queue head, because `_provide()` consumes the FIFO queue from the head, and reaching a deeper order fully removes the shallower ones. However, the provide **self-skip** mechanic re-pushes a provider's own order to the tail instead of consuming it and lets an attacker preserve a dusted order and rotate through the queue:

```
if (state.borrower == provider) {
    ...
    border.queueNodeId = _queue_push(tick.borrowers, state.borrower);
    state.queueHead = tick.borrowers.nodes[state.queueHead].next;
    ...
    continue;
}
```

The attack:

1. From 1000 different addresses, place 1000 legitimate (>= min) borrow orders at the highest APR tick.
2. For each address in turn:
   - a helper provider partially fills the head order down to dust (partial fill keeps it at the head);
   - the head order's own creator calls `provide(MIN_AMOUNT)` , hitting the self-skip, which rotates its dust order to the tail and exposes the next order as the new head.

Repeating stacks 1000 sub-min orders in a single tick, defeating the FIFO protection.

Note that the attacker does not lose principal, only gas (and `protocolFee` on the fills, which is `0` in the reviewed config). The attack is capital-neutral because both sides of every fill are attacker-controlled: when the helper provider fills a dust order, the borrowed assets are credited to the borrower's balance while the provider receives the matching loan claim — both addresses belong to the attacker, so principal just shuffles between their own accounts. Afterwards everything is recoverable: repaying the debt returns the assets to the loan pool, the provider side redeems its loan claim, and the collateral backing each order can be withdrawn once the debt is cleared. During the attack the attacker only bears temporary capital lockup (posted collateral + provided assets) plus the gas to build the clog.

Because `_provide()` traverses ticks from the highest APR down and crossing is bounded per transaction (`maxCrossedOrders` , `uint16` , and block gas — practically ~1,000 orders/tx), a flooded top-

of-book tick forces honest providers to grind through the dust to reach real liquidity beneath it, reverting with an out-of-gas error.

Additionally, a tick composed only of tiny (e.g. 1-wei) orders cannot be swept by an `ExactIn` provide at all: the minimum provide (`MIN_AMOUNT`) can never be fully consumed against sub- `MIN_AMOUNT` total demand, so it reverts with `ProvideNotFulfilled` and clears nothing. Such dust can only be cleared via a deliberate `ExactOut` + preview flow (which a naive provider would not use).

Limited severity

Clearing the clog with ExactOut is about 50× cheaper than creating. This asymmetry makes a sustained DoS economically irrational — a keeper can clear cheaper than an attacker can re-flood.

The persistent, keeper-independent issues are therefore (a) the broken min-amount invariant and (b) the `ExactIn` liveness quirk on dust-only ticks, rather than a profitable attack.

Note that difficulties may arise if gas prices on the network spike sharply during the period when the clog needs to be cleared.

RECOMMENDATION

> We recommend changing the `_provide()` self-skip semantics: as soon as consumption reaches a self order, cut it out and consume orders after it. When finishing with the tick, reinsert the self order at the HEAD of the queue (the opposite of the current behavior). This reduces the attack to at most two consecutive 1-wei orders.
>
> Additionally, set up monitoring for out-of-gas transaction reverts to catch any similar bypass.

M-2 liquidate() has no loss and deadline protection

MEDIUM

FIXED

DESCRIPTION

The liquidator commits to paying `debtAmount`, but the collateral seized is computed inside the tx from the live price, and the discount may vary:

```
uint256 discount = Math.min(
  _getLiquidationDiscountMax(market),
  _getLiquidationDiscountMin(market)
   + UNIT
    - state.healthFactor.min(UNIT)
)
```

and can even drop to zero:

```
function _collateralToSeize...
  ...
  if (debtAmount > collateralValue) return userCollateral;
```

Example:

1. The liquidator sees collateral price = 101,000 USDT, debt = 100,000.
2. The liquidator submits a liquidation tx with debt = type(uint).max, but due to high network demand the tx stalls in the mempool.
3. The price drops to collateral price = 99,000 USDT; debt = 100,000.
4. The tx executes and the liquidator pays `100,000` for collateral worth `99,000` — i.e. takes a `-1,000` loss.

RECOMMENDATION

1. We recommend adding `minDiscountOut` and `deadline` arguments to the function:

```
require(discount >= minDiscountOut)
require(block.timestamp <= deadline)
```

For the line:

```
if (debtAmount > collateralValue) return userCollateral;
```

treat the discount as 0 (so that the `require(discount >= minDiscountOut)` collapses to `require(0 >= minDiscountOut)`).

2. Alternatively, for the `debtAmount > collateralValue` case, recalculate the discount so that the effective `debtAmount` charged to the liquidator is reduced to match the collateral value (rather than treating the discount as zero and reverting). This path, however, introduces another variable dimension to the function: the actual debt size debited from the liquidator may differ from the `debtAmount` they submitted, so callers must bound or preview that outcome explicitly.
3. Document `liquidate()` behavior explicitly — in NatSpec and integration docs — so third-

party liquidators know which outcomes are resolved only at execution time and cannot be inferred from the submitted arguments alone:

- liquidation discount;
- collateral seized;
- effective debt amount debited from the liquidator;
- partial-liquidation constraints (`_minDebtRemainingAfterLiquidation`);
- whether the liquidator can unexpectedly take a loss (repaying more debt than the seized collateral is worth).

M-3 Underlying pool incentive rewards are never claimed

MEDIUM

ACKNOWLEDGED

DESCRIPTION

The underlying pool (Aave or Morpho) may distribute supply-side incentive rewards, but the adapter does not implement any function to claim them.

RECOMMENDATION

> We recommend adding a function to claim these rewards.

CLIENT'S COMMENTARY

All rewards on Monad are currently being distributed through Merkl, and we're in discussions with them about the integration. We won't be adding this at the protocol level.

M-4 Bucket and tick aggregates drift apart and cause DoS

MEDIUM

FIXED

DESCRIPTION

A bucket's `assets` should equal the sum of its 32 ticks, but the bucket and its ticks are rebased against the global index on independent schedules with independent floored divisions and no carried remainder, so the two aggregates drift apart within a couple of calls (a ~90-unit deficit and a growing APR-weighted surplus over a few hundred rounds). Several bucket-minus-tick subtractions are unchecked, so once the aggregate falls below a tick's own share they underflow and revert with an arithmetic panic: a lender who holds nearly the whole bucket can no longer preview a borrow (while a third party still can), and the sole lender in a bucket can neither fully cancel their order nor borrow, reloan or refinance in that market.

RECOMMENDATION

> The root cause is the independent rebasing of the bucket and its ticks against the global index, which lets the two aggregates drift apart. Simply clamping the four bucket-minus-tick subtractions (e.g. `assets = assets > own ? assets - own : 0`, matching the six saturating subtractions already in the contract) would prevent the revert, but it only masks the symptom while leaving the underlying accounting discrepancy in place, and the value it hides has to be absorbed somewhere. Since a robust fix depends on invariants that only the developers can reason about fully, we recommend that they design the remediation themselves, taking care that it addresses the drift at its source and does not introduce new inconsistencies elsewhere in the accounting.

M-5 One protocolFee is applied to two different bases, so the same move costs (1 + fee)× more through reloan than through borrow

MEDIUM

FIXED

DESCRIPTION

`__borrow` computes the protocol fee in two places, on two different quantities, and nothing reconciles them. `ExactOut` takes it upfront on `assetsAmount` — the **net** amount the caller asked for. `ExactIn` takes it at the end on `state.assets` — the **gross** liquidity the order book actually gave up. Both credit the same `_collectedProtocolFee`. A single configured `protocolFee` therefore denotes two different effective rates depending on which branch runs, and `__previewBorrow` reproduces the same split instead of exposing it, so the difference is invisible before execution and no event distinguishes it afterwards.

At a 10% fee the gap is exact on identical volume: the same 1100 of provider liquidity is consumed both ways, and the protocol keeps 100 when `borrow` consumes it (10% of the net 1000) against 110 when `reloan` consumes it (10% of the gross 1100) — a ratio of exactly `1 + fee`, a 9.1% overcharge on the `ExactIn` side. The fee is taken exactly once on the composite paths, so the defect is the basis, not double charging.

Only one entry point is affected. Of the four `__borrow` call sites, `_borrow`, `_executeRefinance` and `_sweep` are all `ExactOut`; `_reloan` is the sole `ExactIn` caller. Ordinary borrowing and the entire rollover path are on the net basis and are not overcharged. The party charged on the gross is the lender who sells a loan claim through `reloan`: it receives 990 for a 1100 claim while the borrower's debt is untouched. That lender has an alternative — resting the claim as a maker rather than taking the immediate exit — which bounds the loss to takers.

RECOMMENDATION

> We recommend charging one base in both branches. Keeping the net basis, the `ExactIn` leg should compute `state.assets · fee / (1 + fee)` instead of `state.assets · fee`, with the same rounding direction as `ExactOut`, so that a given `protocolFee` means one rate on every path and the `preview*` twins agree with execution.

M-6 NoDebtToLiquidate blocks bad-debt socialisation when discounted collateral recovery floors to zero

MEDIUM

FIXED

DESCRIPTION

As part of the M2 liquidation loss/deadline fix, `_collateralToSeize` caps the liquidator's repayment against the discounted value of the borrower's collateral and reverts when that recovery floors to zero:

```
uint256 maxDebtAmount = Math.mulDiv(collateralValue, UNIT - discount, UNIT,
Math.Rounding.Floor);
if (maxDebtAmount == 0) revert NoDebtToLiquidate();
```

When the oracle value of the remaining collateral, after applying the liquidation discount, rounds down to zero, both price-based and post-maturity liquidations revert — even though the position remains `liquidatable` (HF → 0 or past the grace period).

Bad debt is written down only once collateral is fully exhausted:

```
if (state.userCollateral == 0 && state.userDebt > 0) {
    _socializeBadDebt(market, state.badDebtAmount);
}
```

While any residual collateral remains — including dust left by a prior partial liquidation with `debtAmount ≤ collateralValue` — `_socializeBadDebt` never runs. The debt stays booked in `loanPool.totalAssets`, lenders' claims remain overstated, and there is no liquidation path to recognise the shortfall in the exact collateral-goes-to-zero scenario socialisation exists for.

Before the M2 fix, `debtAmount > collateralValue` seized the full collateral while the liquidator still paid the requested `debtAmount` (often absorbing the loss themselves); any unpaid remainder after exhaustion was socialised. The new path refuses the case entirely instead of seizing the residual and writing the shortfall down.

RECOMMENDATION

> Allow collateral exhaustion when `maxDebtAmount == 0`: seize the remaining collateral and socialise the unpaid debt via `_socializeBadDebt`.
>
> Do not simply `return (userCollateral, 0)` without adjusting accounting. `_socializeBadDebt` assumes a repaid amount `R ≥ 1` and asserts `totalAssets > badDebtAmount`; with a sole outstanding debt and `R = 0` that assert fails (`totalAssets == D == B`). Provide an explicit path that either keeps `R ≥ 1`, or permits a full write-down / emptying of the loan pool when discounted recovery is zero.

CLIENT'S COMMENTARY

MixBytes:
The remainder guard closes the partial-liquidation path: a liquidator can no longer leave behind a remainder that is not worth liquidating, and when the collateral is fully seized the residual debt is socialised in the same call, so dust-collateral tails no longer accumulate. The NoDebtToLiquidate revert remains, but it only triggers when the position's entire collateral has become practically worthless relative to the debt asset.

Client:
We accept the assumption that, for the permitted amount of collateral, its price cannot fall so sharply that `maxDebtAmount` rounds down to zero and blocks bad-debt socialization.

2.4 Low

L-1 RWA pitfalls

LOW

ACKNOWLEDGED

DESCRIPTION

Tokenized equities are not supported by the protocol in its current form.

If support for tokenized equities is added later, integrators should account for pitfalls inherited from traditional finance:

1. **Stock splits** — a company may issue additional shares per existing share, or consolidate shares in a reverse split.
2. **Limited trading hours** — traditional exchanges operate on fixed schedules; outside regular sessions, price discovery for the underlying equity can become more volatile or more susceptible to manipulation.
3. **Trading halts** — an exchange may pause trading on a given instrument.

xStocks is a representative example. Its product-specific mechanics include:

- Two tokens: one rebaseable token and one wrapper with a live multiplier.
- Dividends auto-compounded via the live multiplier: the rebaseable balance grows, and the amount of rebaseable tokens represented by one wrapper token increases accordingly.
- Stock splits and reverse splits reflected through the live multiplier.
- Legacy v1 tokens that may be sensitive to donation attacks and should be avoided.

One of the main integration challenges is oracle alignment around splits. If the wrapper token price is derived as **Chainlink per-share price × xStocks Live Multiplier**, a desynchronization window can appear:

a) Chainlink per-share price drops by a split factor, but the xStocks live multiplier has not yet been updated; or
b) the xStocks live multiplier already reflects the split and adjusts the token multiplier accordingly, but the Chainlink feed has not yet incorporated the split.

These risks are informational for future RWA integration and were not exercised against the current deployment configuration.

RECOMMENDATION

> We recommend that, if RWA support is added, the pitfalls above be accounted for when designing the oracle and that the integration be audited before deployment.

CLIENT'S COMMENTARY

Will be taken into account when implementing RWA.

L-2 Underlying-pool utilization DoS

LOW

ACKNOWLEDGED

DESCRIPTION

If utilization in the Aave/Morpho pool where provider liquidity is deployed spikes (up to 100%), any operation that needs to **withdraw** the underlying asset from the lending venue reverts — in particular, `borrow()`, and through it, rollover into another market.

Rollover reborrow goes through `_executeRefinance()` → `__borrow()`, which unconditionally pulls the full borrowed amount from the shared underlying pool at the end of matching:

```
uint256 underlyingPoolAssetsAfter = _withdrawUnderlyingPoolTo(address(this),
state.assets);
```

src/CreditPoolBase.sol

The DoS can happen both spontaneously and deliberately: an attacker can temporarily squeeze a low-liquidity Morpho market, pushing utilization up so that withdrawals are blocked exactly in the maturity window.

Meanwhile, `repay()` does not depend on the underlying pool; it only requires underlying tokens from the borrower, and therefore continues to work:

```
function _repay(address account, address borrower, uint64 market, uint128 amount)
internal {
    ...
    _debitBalance(account, amount, false);
    user.debt[market] -= amount;
    _markets[market].loanPool.balance += amount;
}
```

src/CreditPoolBase.sol

So a borrower with live USDT can repay, but one relying on rollover is blocked — and after the grace period expires they can be liquidated even though the position is healthy.

We considered whether utilization dependence can be removed through internal netting between markets. In the current version this is not feasible, because `repay()` expects underlying tokens: if all underlying is utilized, none is available in other markets either, the debt cannot be repaid, and collateral cannot be moved from one market to another.

RECOMMENDATION

> This is primarily a product/design choice. Possible directions:
>
> 1. **Liquidity buffer** — keep part of provider capital outside Aave/Morpho. This only partially mitigates the issue (up to the buffer size), and it lowers the APR of unfilled provider supply orders (again, by the buffer size). Providers can partially withdraw during high-utilization periods, and rollover becomes partially available up to the buffer.
> 2. **Accept aTokens/morphoShares on `repay()`** instead of underlying — this would unblock rollover for borrowers during a squeeze, but it breaks the expectation that providers receive live underlying strictly at maturity.

> 3. **Do not use external lending venues** — this greatly simplifies the logic and removes the dependency, but provide orders would no longer earn underlying yield.
>
> This is a business-logic dilemma: is it more important that providers always receive live USDT strictly on schedule, or that rollover remains available for borrowers?

CLIENT'S COMMENTARY

Design choice.

L-3 Unbounded getUserMarkets() registry can be inflated by anyone

LOW

ACKNOWLEDGED

DESCRIPTION

`depositCollateral()` does not validate the `market` id and the `getUserMarkets()` registry only ever grows, so anyone can inflate a victim's market set with 1-unit deposits under arbitrary ids. If the array is inflated too much, the caller may hit an out-of-gas error.

RECOMMENDATION

> We recommend one of the following:
>
> - remove the `getUserMarkets()` view and rely on events for reconstructing a user's market set off-chain;
> - introduce pagination, e.g. `getUserMarkets(start, maxCount)`, so consumers can bound the amount of data read in a single call.

CLIENT'S COMMENTARY

Acknowledged

L-4 The rollover keeper controls direction, counterparty, timing, and fee recipient

LOW

ACKNOWLEDGED

DESCRIPTION

The rollover keeper is a permissionless convenience layer, not a trusted party: opt-in and the `keeperFee` are borrower-controlled, and a borrower can always self-execute (`feeRecipient = self`) to avoid the fee. Within that opt-in, however, the keeper's authority is broad. The borrower authorises rollover with a standing Permit2 allowance and a single aggregate `maxDebtFactor` ; every other dimension of the roll is the keeper's choice.

Direction and counterparty. `RolloverSolution.reborrows[]` names the destination market and tick, and the refinance leg hard-codes `maxDebt = type(uint128).max` , so per-leg slippage/APR protection is off:

```
if (factor > maxDebtFactor) revert MaxRolloverFactorExceeded(factor, maxDebtFactor);
```

src/CreditPoolBase.sol

The keeper can therefore rest a provide order at a punitive APR and route `reborrows[]` onto its own tick — choosing, pricing, and taking the other side of the trade. `maxDebtFactor` bounds only aggregate debt growth, not the destination, tick, or counterparty, and it is not validated or bounded at `setRolloverConfig` :

```
function setRolloverConfig(address pool, uint64 market, bool enabled, uint256
maxDebtFactor, uint256 keeperFee) external override {
    _requirePool(pool);
    if (enabled) _checkKeeperFee(keeperFee);
    ...
}
```

src/CreditPoolRolloverExtension.sol

Timing. The `factor` cap is not normalised by the destination's duration, and `_currentFactorFor` raises the allowed debt growth linearly from ~0 at maturity to the full cap at the end of grace, so a patient keeper executes at maximum debt growth right before liquidation opens:

```
return Math.mulDiv(maxDebtFactor, elapsed, gracePeriod, Math.Rounding.Ceil);
```

src/CreditPoolRolloverExtension.sol

Fee recipient and opt-in lifetime. `feeRecipient` is keeper-chosen, the fee is capped only at 100% (`_checkKeeperFee` rejects only `fee > UNIT`), the allowance is bounded by debt rather than a user cap, and the opt-in never expires.

The net effect is that value can be extracted within the borrower-set cap while `maxDebtFactor` is formally respected: the cap bounds the residual exposure but does not eliminate it, and any defect that inflates the debt widens the pull directly. This composes with the shared borrow/liquidation LTV, where a rollover under a loose factor can be sized to land a healthy position just above `UNIT` and liquidated by the same keeper acting as lender and liquidator.

RECOMMENDATION

> Bound each `reborrow` leg with its own `maxDebt`, normalise `factor` by leg duration, let the borrower whitelist destinations and exclude the keeper as counterparty at opt-in, make the factor cap flat or tightening over grace, accept borrower-set `maxDebt` and `deadline`, cap the fee and expire the opt-in when debt reaches zero, and pay the fee to `msg.sender`.

CLIENT'S COMMENTARY

Resolved by a Dutch auction, design choice.

L-5 _creditProvidedLoan credits owner fee shares without registering the market

LOW

ACKNOWLEDGED

DESCRIPTION

Protocol fee loan-shares are credited to `owner` in three places. Two of them call `_regUserMarket(owner, market)` before the credit:

- `__borrow` fee path (`_regUserMarket` + `_creditLoanShares`)
- `_sweep` fee-leg (`_regUserMarket` + `_creditLoan`)

The third — `_creditProvidedLoan` on the user `provide` path — does not:

```
uint256 feeShares = Math.mulDiv(grossShares, fee, grossLoan);
_creditLoanShares(account, market, grossShares - feeShares);
_creditLoanShares(owner, market, feeShares); // no _regUserMarket(owner, market)
```

As a result, after a non-owner provide with `protocolFee > 0`, the owner holds a non-zero loan position but is absent from `getUserMarkets(owner)` / `getMarketUsers`. Funds are not lost (`claim` still works); this is an indexer / explorer blind spot and inconsistent with the other fee paths.

RECOMMENDATION

> We recommend calling `_regUserMarket(owner, market)` in the `account != owner` branch of `_creditProvidedLoan` before `_creditLoanShares(owner, ...)`.

CLIENT'S COMMENTARY

Acknowledged

L-6 _previewProvide does not model LoanPool burns across Reloan fills, so preview understates fillability

LOW

FIXED

DESCRIPTION

Live `_provide` burns the reloan seller's loan claim on each Reloan fill via `_burnLoanAssets`, which updates `LoanPool.totalAssets`, `LoanPool.totalShares`, and the seller's shares before the next order is sized:

```
} else if (state.orderType == BOrderType.Reloan) {
    _burnLoanAssets(state.borrower, state.market, state.debt);
}
```

`_previewProvide` never applies that burn. It only advances `collectedLoan` / `remainingAssets`, while each Reloan fill still caps availability with `__loanOf` against the unchanged storage pool:

```
if (state.orderType == BOrderType.Reloan) {
    uint128 borrowerLoan = __loanOf(state.market, state.borrower);
    availableDebt = borrowerLoan < border.amount ? borrowerLoan : border.amount;
    ...
}
```

After floored socialization (e.g. 1 wei of bad debt), `Σ loanOf < totalAssets`. Burning one seller's entire position concentrates the orphaned dust into the remaining shareholders' redeemable balances, so a later Reloan fill in the same live pass can consume more than preview observed for that seller.

Concrete case with equal shares, `LoanPool.totalAssets = 4e18`, Alice loan `2e18`, Bob loan `2e18`, both resting Reloan orders `2e18 @ 0%`, then `badDebt(1)`:

1. After socialization: `totalAssets = 4e18 − 1`, `loanOf(Alice) = loanOf(Bob) = 2e18 − 1`. Orders remain nominally `2e18`.
2. Provider ExactIn for `4e18 − 1` assets.
3. **Preview:** both fills capped at `2e18 − 1` → reports consumed `4e18 − 2`.
4. **Live:** Alice burns all shares (`2e18 − 1`); pool becomes `totalAssets = 2e18` with Bob holding all remaining shares → `loanOf(Bob) = 2e18`; Bob fills `2e18`. Total consumed `4e18 − 1`.

So:

- preview consumed = `4e18 − 2`
- live consumed = `4e18 − 1`

For ExactOut the same gap yields a false negative: preview reports the target unreachable while execution reaches it. The absolute discrepancy is dust-scale (here 1 wei; with `N` equal lenders after 1 wei bad debt, up to `N − 1` wei), but it is a real preview/live divergence, not a rounding artifact in a single fill. This is the inverse of Low finding 7 (Borrow debt shadow): that overstates fillability; this understates it for sequential Reloan fills.

RECOMMENDATION

> We recommend tracking a shadow LoanPool (`totalAssets` / `totalShares`) and per-seller share balances inside `_previewProvide`, applying the same burn math as `_burnLoanAssets` after each Reloan fill so subsequent `__loanOf` caps match live exchange-rate recovery.

L-7 _previewProvide does not accumulate borrower debt across fills, so ExactOut preview can overstate fillability

LOW

ACKNOWLEDGED

DESCRIPTION

The provide path documents a preview that mirrors live matching:

```
// execution, with a preview path that mirrors live matching.
```

Live `_provide` updates the borrower's booked debt after each Borrow fill, so a later fill of the same borrower is health-checked against the increased total:

```
if (__getHealthFactor(state.market, state.borrower, state.priceCache, state.debt, 0,
_borrowLTV) < UNIT) {
    // invalidate order
}
...
borrower.debt[state.market] += state.debt;
```

`_previewProvide` runs the same `__getHealthFactor` call with `debtIncrease = state.debt`, but never models the debt written by earlier fills in the same preview pass — it only advances `collectedLoan` / `remainingAssets`:

```
if (__getHealthFactor(state.market, state.borrower, state.priceCache, state.debt, 0,
_borrowLTV) < UNIT) {
    state.queueHead = tick.borrowers.nodes[state.queueHead].next;
    continue;
}
...
state.collectedLoan += netLoan;
state.remainingAssets -= state.consumed;
```

This is reachable whenever `maxActiveBorrowOrders > 1` (the suite itself uses `params.maxActiveBorrowOrders = 3`). Placement health-checks each resting Borrow order against storage debt alone and does not reserve capacity for other open orders of the same borrower, so several individually healthy orders can sit on different APRs. Matching walks APR ticks from high to low, so successive fills of that borrower are common in one provide.

Concrete case under `borrowLTV = 50%`, collateral `300`, debt capacity `150`, storage debt `0`:

1. Borrower rests `100` assets @ `10%` → `110` debt and `100` assets @ `0%` → `100` debt. Each placement passes (`110 ≤ 150`, `100 ≤ 150`).
2. Provider calls ExactOut with `maxAssets = 200`, `loanAmount = 210`.
3. **Preview:** fill #1 against `0 + 110`, fill #2 against `0 + 100` → reports `loan = 210`, `assets = 200`.
4. **Live:** fill #1 books `debt = 110`; fill #2 checks `110 + 100 = 210 > 150`, invalidates the order, then reverts `ProvideMinLoanNotReached(110, 210)`.

Integrators that size or simulate `provideExactOut` / `provideExactIn` from `previewProvide*` therefore receive a false positive on fillability: preview claims a target is fully reachable while execution reverts with the same parameters.

RECOMMENDATION

> We recommend tracking shadow debt per borrower inside `_previewProvide` for fills already accounted in the current pass, and feed `storageDebt + shadowDebt + candidateDebt` into the health check (mirroring live `borrower.debt += state.debt` between fills). Optionally shadow remaining order amounts the same way so partial-fill revisits stay consistent with execution.

CLIENT'S COMMENTARY

Acknowledged for `maxActiveBorrowOrders > 1`; when `maxActiveBorrowOrders = 1`, preview always matches provide.

L-8 Borrow-order invalidation depends on counterparty size and can cancel a still-collateralized remainder

LOW

FIXED

DESCRIPTION

On the provide matching path, solvency of a resting Borrow order is checked against the debt of the **current fill**, not against the full remaining order and not against a max healthy partial fill. `_initBOrderProviding` sets `consumed = min(providerRemaining, border.amount)` and derives `state.debt` from that slice:

```
state.consumed = state.remainingAssets < state.borderAssets ? state.remainingAssets :
state.borderAssets;
state.debt = _debtFor(state.consumed, APR, state.debtDuration, Math.Rounding.Ceil);
```

If that candidate fill would push `__getHealthFactor(..., state.debt, ...)` below `UNIT`, the matcher invalidates the **entire** order (`border.amount`), clears `queueNodeId` , and continues:

```
if (__getHealthFactor(state.market, state.borrower, state.priceCache, state.debt, 0,
_borrowLTV) < UNIT) {
    _recordBorrowOrderInvalidation(state.changes, state.borrower, APR,
border.amount);
    border.queueNodeId = 0;
    ...
    continue;
}
```

So the same resting order can partially fill or be fully cancelled depending only on how large the next provider is.

Concrete case under `borrowLTV = 50%` , collateral capacity `150` debt units, storage debt `100` , resting Borrow order `100` :

1. Provider brings `40` → check `100 + 40 = 140 ≤ 150` → order is partially filled.
2. Provider brings `100` → check `100 + 100 = 200 > 150` → the full `100` order is invalidated, even though roughly `50` of it was still safely collateralized.

This is reachable with the default `maxActiveBorrowOrders = 1` :

1. Borrower places a Borrow order (placement health-checks `storageDebt + full order debt`).
2. Borrower later takes debt via `borrow()` against p-order liquidity.
3. `_borrow` books `debt[market] += debt` and checks only storage debt — it does **not** reconcile or cleanup the resting Borrow order:

```
_users[account].debt[market] += debt;
_requireHealthyDebt(market, account, priceCache);
```

4. The resting order becomes only partially backed; the next foreign `provide()` decides whether it survives as a partial fill or is wiped.

The same bifurcation also appears after an oracle move that leaves residual capacity below `border.amount` but above zero: a small provide can still fill, a large one cancels everything.

This yields a griefing / order-cancellation primitive. The attacking provide need not revert or forfeit assets: after invalidating the victim tick, matching continues down the book, so the attacker can take a normal loan claim against another healthy Borrow order (e.g. victim @ 10% APR invalidated, then fill a healthy order @ 5%). Only a successful transaction is required.

The codebase already mixes two models elsewhere: `_cleanupBorrowOrders` (collateral withdraw / refinance) health-checks the **full** `border.amount`, while provide matching health-checks only the candidate fill and then deletes the whole order. That inconsistency is the root of the issue.

RECOMMENDATION

> We recommend choosing a single invariant and implementing matching consistently with it:
>
> - If a Borrow order must remain fully collateralized at all times, the provide health check should evaluate the full remaining `border.amount` (independent of the current provider size), and undersized / underwater orders should be cleaned up on any path that increases booked debt (including `borrow()`), not only on collateral withdraw.
> - If partially backed resting orders are allowed, the matcher must compute the maximum healthy partial fill against residual capacity and must not delete the remainder when a larger counterparty arrives.
>
> The current behavior is a mixture of both models and makes order survival depend on counterparty size.

[3 ABOUT MIXBYTES]

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL	AUDITS DELIVERED	WEB3 PROTOCOLS
$50B+	300+	80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io